

Algorithms for Data Science

Cheatsheet for Exercises 2

Recall the definition of **asymptotic notation**:

- $f(n) = O(g(n))$ if there exist constants $C > 0$ and $n_0 \geq 0$ such that, for all $n \geq n_0$, $f(n) \leq C \cdot g(n)$.
- $f(n) = \Omega(g(n))$ if there exist constants $C > 0$ and $n_0 \geq 0$ such that, for all $n \geq n_0$, $f(n) \geq C \cdot g(n)$.
- $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

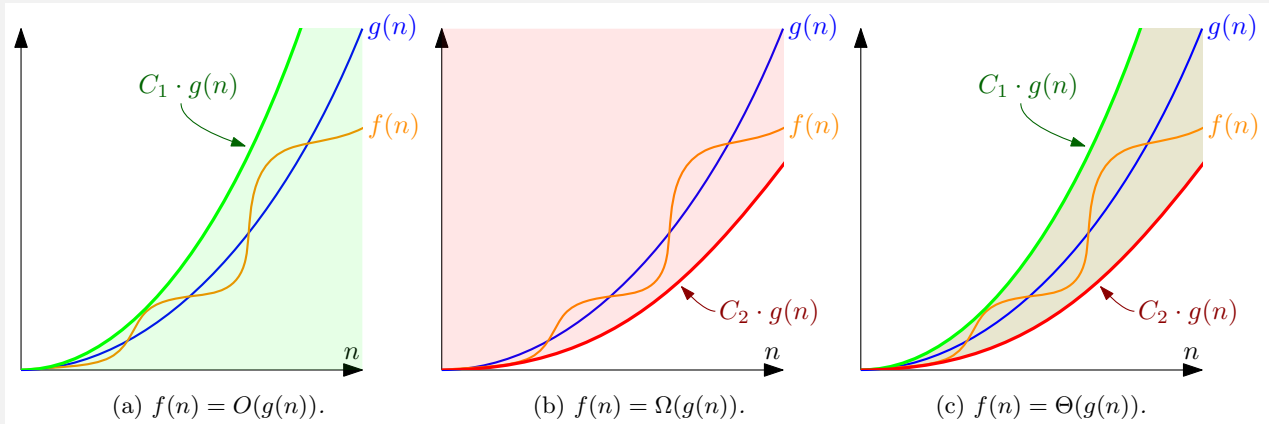
Equivalently, $f(n) = \Omega(g(n))$ if $g(n) = O(f(n))$ —see Task 2.3(b)

Intuitively:

- $f(n) = O(g(n))$ if g is an “**upper bound**” on f .
- $f(n) = \Omega(g(n))$ if g is a “**lower bound**” on f .
- $f(n) = \Theta(g(n))$ if f and g are asymptotically **equal**.

Furthermore: f is asymptotically **faster** than g if f is not a “lower bound” on g , that is, if $g(n) \neq \Omega(f(n))$.

Graphically:



Explanation:

- In this example, there is a fixed number C_1 such that $f(n)$ lies below the function $C_1 \cdot g(n)$ for all $n \geq 0$ (that is, we can set $n_0 = 0$). Thus, we have $f(n) = O(g(n))$.
- Similarly, $f(n) = \Omega(g(n))$ as there is a fixed number C_2 such that $f(n)$ lies above $C_2 \cdot g(n)$ for all $n \geq 0$.
- Since $f(n)$ lies in the “corridor” defined by C_1 , C_2 and $g(n)$, we have $f(n) = \Theta(g(n))$.

Ensure you understand this: The faster an algorithm is, the slower its running time is growing!
(Since a faster algorithm takes less time, it's running time is growing more slowly.)

In our course:

- $\log$ always means $\log_2$, and
- *running time* always means the *worst-case* running time (if not otherwise noted).

Recall that $\log_b a$ is defined as the x satisfying the equality $b^x = a$.

Def.: A **simplification** of $f(n)$ is a $g(n)$ that looks as “simple” as possible and that satisfies $f(n) = \Theta(g(n))$.

Simplification Rule: (*Works only if we have constant many terms.*)

1. Replace constant factors with 1,
2. transform complicated terms to simpler forms, and
3. remove any term that grows slower than another.

Use the rules below to simplify terms and compare their growth rate.

Example: $\Theta(9999 + 0.1 \cdot 2^n + 100 \cdot n^2 \cdot n) = \Theta(1 + 1 \cdot 2^n + 1 \cdot n^2 \cdot n) = \Theta(1 + 2^n + n^3) = \Theta(1 + 2^n + n^3) = \Theta(2^n)$

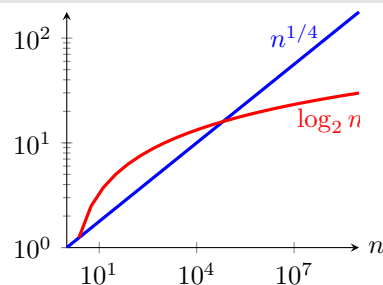
“LOG < POL” Rule:

Any logarithmic function grows slower than any polynomial function.

Formally: If a , b and c are positive constants, then there is a constant n_0 , such that for all $n \geq n_0$, we have

$$a \log_b n < n^c \quad \text{and} \quad (\log_b n)^a < n^c .$$

For example, $\log_2 n < n^{1/4}$ for sufficiently large n . Without the rule above, this fact is not obvious! If you try out different values for n , even as large as $n = 10000$, you might get the impression that $\log_2 n$ is growing faster. However, for all $n \geq 65537$, the function $\log_2 n$ is smaller than $n^{1/4}$.



“POL < EXP” Rule:

Any polynomial function grows slower than any exponential function.

Formally: If a and $b > 1$ are constants, then there is a constant n_0 , such that for all $n \geq n_0$, we have

$$n^a < b^n .$$

For example, $n^{100} < 1.01^n$ for sufficiently large n .

Transformation Rules:

For any a, b, c , we have

- $a^b \cdot a^c = a^{b+c}$ and $(a^b)^c = a^{b \cdot c}$,
- $\log_a(b \cdot c) = \log_a b + \log_a c$ and $\log_a(b^c) = c \cdot \log_a b$ (for positive variables)
- $a = b^{\log_b a}$ definition of the logarithm (for positive variables)
- $\Theta(\log_a c) = \Theta(\log_b c) = \Theta(\log c)$ (if a and b are constants).